Information and Coding — Trabalho Prático n.º 1

Relatório 1: Ferramentas de áudio e codecs com e sem perdas

Duarte Lourenço (n.º mec. 114421) Francisco Matos (n.º mec. 113726)

Universidade de Aveiro — DETI
Outubro de 2026

Resumo

Este relatório descreve as quatro ferramentas de análise e manipulação de áudio da Parte I, um codec sem perdas e um codec com perdas baseado na MDCT. Nos 20 ficheiros de treino o codec sem perdas obtém uma taxa de compressão total de 2,28, contra 2,57 do `flac -8`, com o mesmo tempo de codificação, 4,8 MiB de memória que não dependem da duração do áudio e saída igual ao original byte a byte. O codec com perdas obtém 45,97 dB de SNR médio a 128 kbit s^{-1}, contra 21,64 dB do MP3 e 25,38 dB do AAC, diferença que o SNR favorece por construção, por ser a grandeza que o nosso codec minimiza. Como a avaliação final corre num conjunto cego, nenhum parâmetro do codec sem perdas é fixado à mão, e os dois parâmetros fixos do codec com perdas foram escolhidos em zonas planas da curva de desempenho.

1 Introdução

O trabalho divide-se em três partes. A primeira constrói quatro ferramentas de análise e manipulação de áudio, que geram histogramas dos canais de um ficheiro WAV, reduzem o número de bits usados em cada amostra, medem a diferença entre uma versão processada e o original, e aplicam efeitos ao sinal. A segunda desenvolve um codec sem perdas com o melhor desempenho possível em taxa de compressão, tempo e memória, com reconstrução exata do original e comparação com o estado da arte especializado (`flac`) e de uso geral (`zstd`, `xz`). A terceira desenvolve um codec com perdas com o melhor compromisso entre débito e distorção, obrigatoriamente baseado na DCT, que é a única restrição imposta pelo enunciado, comparado com o MP3 e o AAC.

Nos 20 ficheiros de treino o codec sem perdas obtém uma taxa de compressão total de 2,28 contra 2,57 do `flac -8`, fica a par dele no tempo de codificação, usa 4,8 MiB de memória independentemente do tamanho do ficheiro e reproduz o original byte a byte. O codec com perdas obtém entre 34 e 58 dB de SNR a 128 kbit s^{-1}, conforme a coleção, mas como o SNR mede o erro quadrático, que é a grandeza minimizada pelo codec, a comparação com o MP3 e o AAC avalia objetivos distintos e não permite concluir qual apresenta melhor qualidade sonora.

A avaliação final corre num conjunto cego de dez faixas, cinco de cada coleção, o que motivou uma decisão de fundo. Nenhum parâmetro do codec sem perdas é fixado manualmente, porque o modo de canais, a ordem do preditor e o número de partições de Rice são determinados em cada bloco pela minimização do custo em bits. O relatório segue a ordem do trabalho, com a Secção 2 nas escolhas tecnológicas e na metodologia de medição, a Secção 3 nas ferramentas da Parte I, as Secções 4 e 5 nos dois codecs, e as Secções 6 e 7 nas conclusões e no manual de utilização.

2 Arquitetura e escolhas tecnológicas

2.1 Linguagens, bibliotecas e compilação

Cada ficheiro de teste contém milhões de amostras por canal, e o codificador sem perdas avalia em cada bloco dois modos de canais, quatro ordens de preditor e vários números de partições, o que resulta em dezenas de passagens sobre as mesmas amostras. Foi essa carga que ditou o uso de C++ no processamento. Mesmo na tarefa mais leve do trabalho, o histograma, são 0,093 s de CPU em C++

Contribuição dos autores, 50 % Duarte Lourenço e 50 % Francisco Matos.

contra 0,226 s em Python puro e 0,096 s com a biblioteca `numpy`, e no codificador a diferença acentua-se, porque as decisões de cada bloco são sequenciais e não se vetorizam. A segunda razão é o controlo da representação ao nível do bit, que em Python não existe, por os inteiros terem precisão arbitrária e não haver um tipo de 32 bits com largura garantida. A norma usada é o C++20, que garante que o deslocamento à direita de um inteiro negativo é aritmético, condição necessária para o codificador e o descodificador aplicarem a mesma convenção de divisão inteira.

O Python é usado nos gráficos, nas varreduras de parâmetros e no recálculo das métricas em `numpy`, que é uma implementação independente e, por chegar aos mesmos valores, reforça a confiança nos números apresentados. A compilação usa CMake, que fixa a norma C++20 e compila em `Release` com `-O2` e avisos ativos, e foi verificada a partir de um clone limpo numa máquina diferente da de desenvolvimento, sem erros nem avisos, o que importa porque a avaliação é automática e corre noutro sistema.

A `libsndfile` é usada apenas pelo `wav_hist` e pelo `wav_effects`. Os restantes programas, incluindo os dois codecs, leem e escrevem WAV com código próprio, e a razão é a memória, que é critério de avaliação, porque a biblioteca carrega libFLAC, libvorbis, libopus e libmpg123 no processo sem que nenhuma seja necessária para ler um WAV. O leitor próprio foi validado contra ela em 14 ficheiros de 8, 16, 24 e 32 bits, com ficheiro comprimido bit a bit igual. O código está em `https://github.com/Duarte-Lourenco/Grupo-IC---DL_FM_GG`, onde o `README` descreve a estrutura do repositório.

2.2 Conjunto de dados de teste

O conjunto de treino são 20 faixas, 10 do DSD100, mixagens densas de banda larga, com 300 s e 50,47 MiB, e 10 do EBU SQAM, material crítico com instrumentos solo, voz e sinais de teste, com 409 s e 68,81 MiB, o que perfaz 709 s e 119,27 MiB. Todas em WAV PCM de 16 bits, estéreo, 44,1 kHz, normalizadas pela equipa docente, com cabeçalho canónico de 44 bytes e sem blocos de metadados.

A avaliação final corre num conjunto cego de 10 faixas, 5 de cada coleção, o que exclui afinar parâmetros ao conjunto de treino. As duas coleções têm compressibilidade muito diferente, pelo que os resultados deste relatório são dados também por coleção e não apenas em média.

2.3 Metodologia de medição e verificação

Todas as medições foram feitas na mesma máquina e na mesma sessão: AMD Ryzen 7 6800H com 16 núcleos lógicos, 14,9 GiB de RAM, Linux 7.0.0, g++ 13.3.0 e CMake 3.28.3, com os programas compilados em `Release` com `-O2`; as comparações usam `flac` 1.4.3. Os tempos e o pico de memória vêm do melhor de três execuções.

O *governor* do CPU foi fixado em `performance`. Em `powersave` o processador corria a 1,1 GHz a 1,36 GHz e uma ronda inteira de medições chegou a inverter a conclusão sobre qual das versões do codec era mais rápida.

As taxas de compressão aparecem em duas convenções. A taxa *total* é a soma dos tamanhos originais a dividir pela soma dos comprimidos e a taxa *média* é a média aritmética das taxas de cada ficheiro. No nosso codec dão 2,28 e 2,49, e a divergência vem de as taxas por ficheiro variarem entre 1,42 e 5,28 e de os ficheiros terem entre 3,7 MiB e 12,5 MiB. A média dá o mesmo peso a todos, a total pesa cada um pelo seu tamanho. Salvo indicação em contrário, as tabelas deste relatório usam a taxa total, e as que apresentam as duas identificam-nas.

A verificação dos resultados assenta em quatro caminhos independentes. O primeiro é a reconstrução de ida e volta validada com `cmp`, que exige igualdade byte a byte e não apenas igualdade das amostras. O segundo é o recálculo dos histogramas, das entropias e das métricas de erro em `numpy`, que constitui uma segunda implementação, independente da primeira. O terceiro são os testes unitários `test_midside`, `test_bitstream`, `test_golomb` e `test_mdct`, que verificam propriedades (reversibilidade da transformação MID/SIDE, ida e volta do mapeamento zigzag, conservação de energia na MDCT) em vez de valores tabelados, pelo que permanecem válidos para qualquer entrada. O quarto é uma bateria de 25 casos-limite do codec sem perdas, que cobre ficheiros mono e de 8, 16, 24 e 32 bits, ficheiros de 1, 2 e 5 *frames*, bloco `data` com número ímpar de bytes, contentores com metadados,

cabeçalhos que declaram mais dados do que existem, blocos de 16 a 65 535 amostras, silêncio total, canais idênticos, um canal em silêncio, tom puro e onda saturada, bem como a recusa de ficheiros comprimidos truncados ou aleatórios. Os 25 casos passam.

O `wav_cmp` e os dois codecs calculam o erro com o mesmo código, pelo que os valores apresentados nas três partes são comparáveis entre si.

3 Parte I — Ferramentas de processamento de áudio

3.1 `wav_hist`: histogramas e análise

O `wav_hist` conta as ocorrências de cada valor e escreve um CSV; o gráfico é feito em Python, pelo critério da Secção 2. Analisam-se quatro canais: L, R, MID $= \lfloor (L+R)/2 \rfloor$ e a diferença média $(L-R)/2$, que é a pedida no enunciado e não coincide com o SIDE do codec da Secção 4, que é $L-R$.

As contagens podem ser agrupadas em *bins* de 2^k valores por truncagem dos k bits baixos; a Tabela 1 usa $k=0$, porque agrupar símbolos reduz a entropia, e a Figura 1 usa $k=8$. A divisão por dois é inteira, mas a reconstrução continua exata: $L+R$ e $L-R$ têm a mesma paridade, pelo que $L+R = 2\,\mathrm{MID} + (\mathrm{SIDE}\ \&\ 1)$. Isto responde à nota de rodapé 2 do enunciado e foi verificado em milhares de pares (L,R) e em todas as amostras dos ficheiros de teste.

As distribuições têm pico em zero e decaimento aproximadamente exponencial, forma que fundamenta a escolha de Golomb/Rice na Secção 4.3. A diferença média tem sempre menor desvio padrão do que L e R, com margens de 1,6 a 129 vezes conforme o material, e a transformação poupa 1,59 bits por *frame* no DSD100 e 2,26 no SQAM. O par MID/SIDE sai mais barato nos 20 ficheiros, sem exceção; ainda assim o modo é decidido por bloco, porque a entropia por ficheiro é uma média que não distingue os trechos em que os canais divergem.

Estas entropias são de ordem zero e tratam cada amostra como independente das anteriores, pelo que não veem a correlação temporal. Embaralhar o ficheiro não altera o histograma, mas destrói a compressibilidade. A última coluna da Tabela 1 permitiria 1,25 no DSD100 e 1,46 no SQAM, e o codec da Parte II obtém 1,77 e 2,90; a diferença mede o que vem da modelação temporal e não contraria o limite de Shannon, que se aplica ao modelo com que foi calculado.

Tabela 1: Entropia de ordem zero (bits/amostra) e desvio padrão por canal, nos 20 ficheiros de treino. São dadas as faixas de menor e de maior entropia de cada coleção e as médias respetivas. O quarto canal é a diferença média $(L-R)/2$, não o $L-R$ do codec. A última coluna é a taxa de compressão que esta entropia permitiria se as amostras fossem independentes.

Ficheiro	H_L	H_R	H_{MID}	$H_{(L-R)/2}$	σ_L	$\sigma_{(L-R)/2}$	$16/H_L$
`090_mix`	11,30	11,72	11,31	10,61	724	463	1,42
`098_mix`	14,35	14,37	14,27	12,82	5150	1761	1,12
`sqam_40`	9,84	9,85	9,67	8,76	755	414	1,63
`sqam_14`	12,08	12,18	12,01	11,06	3923	1869	1,32
Média DSD100	12,84	12,90	12,76	11,39	3015	1062	1,25
Média SQAM	10,92	10,93	10,88	8,71	2706	672	1,46

3.2 `wav_cmp`: métricas de erro

Para cada canal e para o MID, o programa calcula o erro quadrático médio (norma L^2), o erro máximo por amostra (norma L^∞) e a relação sinal-ruído:

$$\mathrm{SNR} = 10 \log_{10} \frac{\sum_n x[n]^2}{\sum_n \left(x[n] - \tilde{x}[n]\right)^2} \ \mathrm{dB} \tag{1}$$

As três métricas são complementares e nenhuma dispensa as outras. O MSE descreve o comportamento médio, mas dilui o pior caso entre milhões de amostras; o L^∞ expõe a amostra isolada mais errada, audível como um estalido e que o MSE esconde; o SNR normaliza o erro pela energia do sinal, o que permite comparar ficheiros com volumes diferentes. Os valores são dados para L, para R e para o

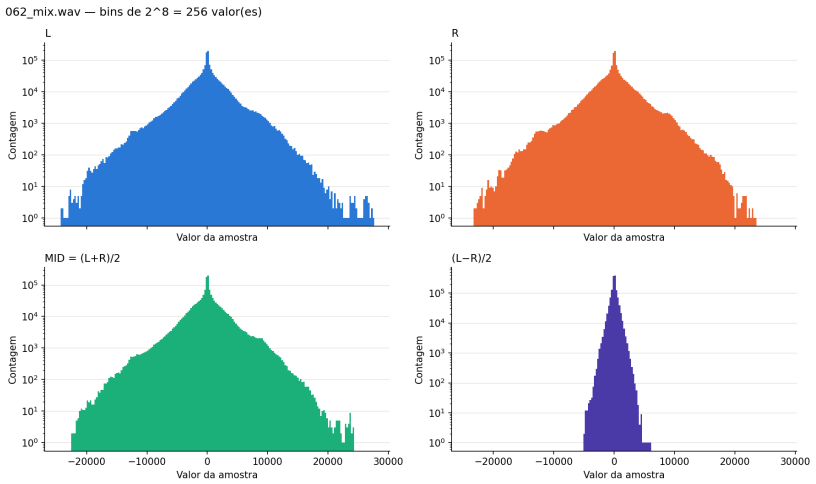


Figura 1: Histogramas de L, R, MID e $(L-R)/2$ do `062_mix`, em *bins* de 2^8 valores e escala logarítmica em y, onde o decaimento linear dos flancos corresponde a contagens com decaimento exponencial. Com o mesmo eixo das abcissas, a diferença média ocupa cerca de um quinto da gama de L e R.

MID, que é o canal médio pedido no enunciado. A acumulação é feita em vírgula flutuante de dupla precisão, porque a soma dos quadrados excede a gama de um inteiro de 32 bits.

Este programa é o instrumento de medida das Partes II e III: um erro aqui não afetaria uma secção, afetaria todos os números do relatório. É essa a razão de as suas saídas terem sido recalculadas de forma independente em `numpy` (Secção 2.3).

3.3 `wav_quant`: quantização escalar uniforme

Para reduzir uma amostra de 16 para b bits, o programa usa um quantizador uniforme com passo $\Delta = 2^{16-b}$, com índice $q = \lfloor s/\Delta \rfloor$ e amostra reconstruída $\tilde{s} = q\Delta + \Delta/2$, ou seja o ponto médio do intervalo de quantização. A alternativa mais simples, truncar ($\tilde{s} = q\Delta$), produz um erro sempre do mesmo sinal, entre 0 e $\Delta - 1$, e com o ponto médio o erro fica centrado em zero. O erro máximo medido confirma-o em todos os ficheiros, sendo exatamente $\Delta/2$ com o ponto médio e $\Delta - 1$ a truncar (Tabela 2). Em média o ponto médio ganha 4,4 a 4,6 dB de SNR (5,0 a 5,5 dB no DSD100 e 3,0 a 3,8 dB no SQAM, pela razão explicada abaixo), e é por isso o modo por omissão, ficando a truncagem disponível como opção. Como $\tilde{s}$ fica sempre dentro do intervalo da amostra original, a saída nunca excede a gama do formato e não é necessária saturação.

A Tabela 2 compara as medições com o modelo teórico. Entre 12 e 8 bits o SNR desce 6,07 dB por bit, próximo do valor esperado de $20\log_{10}2 \approx 6{,}02$ dB. O MSE coincide com $\Delta^2/12$ nos ficheiros sem silêncio (razão de 1,01 no `098_mix`, que não tem amostras nulas), mas fica acima nos restantes. A causa é o silêncio digital, porque o quantizador de ponto médio não tem o nível zero e cada amostra nula é reconstruída como $\Delta/2$, com erro quadrático $\Delta^2/4$, três vezes o valor médio. Com uma fração p de amostras nulas, o MSE esperado passa a $(1+2p)\,\Delta^2/12$. A 12 bits esta previsão acerta na razão medida com erro médio de 1,2 % (máximo 5,4 %) nos 20 ficheiros, que vão de 0 a 24 % de amostras nulas, e as gravações do SQAM, com mais silêncio, são as que mais se afastam de $\Delta^2/12$. É também por isso que o ponto médio ganha menos no SQAM, porque no silêncio truncar reconstrói o zero exatamente. Como o MSE e o SNR são calculados pelo `wav_cmp`, este acordo quantitativo com a teoria valida os dois programas. Abaixo de 6 bits o modelo deixa de se aplicar ao sinal, porque o passo se torna comparável à amplitude da música e o erro deixa de se distribuir uniformemente no intervalo.

Por fim, o ficheiro produzido continua a ser um WAV de 16 bits e não diminui de tamanho, porque há apenas 2^b valores distintos mas cada um continua a ocupar 16 bits. A redução só se concretiza quando um codificador entrópico atua a seguir e gasta poucos bits nos valores mais frequentes, que é precisamente a combinação usada no codec com perdas da Parte III, aplicada aos coeficientes da MDCT em vez de às amostras.

		MSE				SNR (dB)	
Bits	Δ	teórico ($\Delta^2/12$)	medido	razão	L^∞ medido	ponto médio	truncar
16	1	0	0	—	0	∞	∞
12	16	21,3	25,9	1,21	8	53,93	49,53
10	64	341,3	423,6	1,24	32	41,79	37,16
8	256	5461	6969	1,28	128	29,64	25,02
6	1024	87 381	121 816	1,39	512	17,23	12,88
4	4096	1 398 101	2 229 191	1,59	2048	4,64	0,62
2	16384	22 369 621	47 412 537	2,12	8192	−8,66	−11,96

Tabela 2: Quantização nos 20 ficheiros de treino (canais L e R). MSE e SNR médios, L^∞ máximo, com reconstrução no ponto médio. A razão entre o MSE medido e o teórico vem do silêncio digital, explicado no texto.

3.4 `wav_effects`: efeitos de áudio

O `wav_effects` aplica quatro efeitos, todos descritos por equações às diferenças sobre as amostras. O eco simples é $y[n] = x[n] + a\,x[n-D]$, uma única repetição atenuada. Os ecos múltiplos usam realimentação, $y[n] = x[n] + a\,y[n-D]$, o que exige $|a| < 1$ para a saída não divergir. A modulação de amplitude é $y[n] = x[n]\,(1 + m\cos(2\pi f n/f_s))$. O quarto efeito é um atraso variável no tempo, em que a posição a ler cai entre duas amostras e é obtida por interpolação linear. Os atrasos são indicados em milissegundos e as frequências em hertz, convertidos para amostras pela taxa de amostragem do ficheiro.

A implementação foi validada com um impulso: à entrada de um único valor não nulo, a saída do eco com realimentação apresenta impulsos em D, $2D$, $3D\ldots$ com amplitudes a, a^2, $a^3\ldots$, o que confirma o atraso e o fator de atenuação sem recorrer à audição.

4 Parte II — Codec de áudio sem perdas

O codec remove redundância em andares reversíveis, pelo que a saída do descodificador é igual à entrada do codificador por construção e não por aproximação. Nenhum parâmetro da codificação é fixado à mão. O modo de canais, a ordem do preditor, o número de partições de Rice e o k de cada uma são escolhidos bloco a bloco, por minimização do custo real em bits.

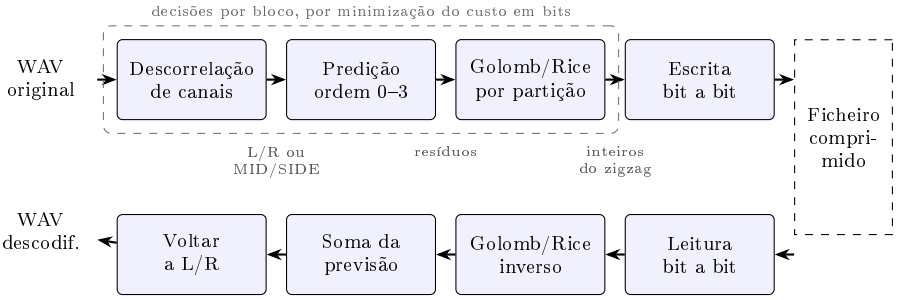


Figura 2: Codec sem perdas, com o codificador em cima e o descodificador em baixo. Todos os andares são reversíveis, ao contrário da Figura 3.

4.1 Formato do ficheiro e preservação do contentor

O cabeçalho do ficheiro comprimido ocupa 29 bytes e guarda a marca de formato, os parâmetros do WAV original, o número de *frames*, o tamanho do bloco e os comprimentos dos dois troços de contentor. O número de *frames* é indispensável, porque o último byte do fluxo é completado com zeros e sem ele o descodificador não sabe onde parar de ler resíduos. Cada bloco tem o seu próprio cabeçalho, com um bit para o modo de canais, ausente em mono, e por canal dois bits de ordem, três bits para o expoente do número de partições e cinco bits por cada parâmetro k. Em blocos de 4096 amostras isso custa 21 bits no mínimo, com uma partição por canal, e 1291 bits no limite de 128 partições. O

número de partições não é fixo e entra na mesma minimização de custo, com os cinco bits de cada k contabilizados, pelo que um valor mais alto só é escolhido quando paga o seu próprio cabeçalho.

A preservação do contentor foi o problema mais instrutivo desta parte. A primeira versão escrevia a saída com a `libsndfile`, que produz um cabeçalho mínimo, e o `wav_cmp` dava erro nulo nas amostras enquanto o `cmp` falhava. Foi essa contradição que localizou o problema fora do codec, nos blocos de metadados que se perdiam, e a solução foi guardar os bytes anteriores e posteriores ao bloco `data` e reescrevê-los literalmente. A visibilidade desta propriedade é, no entanto, limitada. Os ficheiros oficiais foram normalizados e têm cabeçalho canónico de 44 bytes sem metadados, pelo que o `flac` também passa o `cmp` neles. O codec distingue-se apenas em ficheiros com metadados, onde guardar o contentor custava 5766 bytes contra os 44 atuais.

4.2 Descorrelação entre canais e predição temporal

Num ficheiro estéreo os dois canais tomam valores próximos, pelo que o codec os representa pela parte comum e pela diferença, com o MID igual à divisão inteira de $L + R$ por dois e o SIDE igual a $L - R$. A divisão descarta o bit menos significativo de $L + R$ e ainda assim a reconstrução é exata, porque $L + R$ e $L - R$ têm sempre a mesma paridade, pelo que $L + R = 2\,\mathrm{MID} + (\mathrm{SIDE} \,\&\, 1)$. O par MID e SIDE custa menos do que o par L e R nos 20 ficheiros de treino, sem exceção, mas o modo é decidido em cada bloco, ao custo de um bit, porque a entropia por ficheiro é uma média que não distingue os trechos em que os canais divergem.

Em vez da amostra transmite-se o erro de previsão, que o descodificador soma à previsão refeita a partir das amostras que já reconstruiu. São usados quatro preditores polinomiais fixos, de ordem 0 a 3, com coeficientes binomiais de sinais alternados, e a ordem p anula qualquer troço do sinal que seja polinomial de grau inferior a p. Não se vai além da ordem 3 porque a diferenciação amplifica o ruído por um fator igual à soma dos quadrados dos coeficientes (1, 2, 6 e 20), o que encarece o piso de ruído em cerca de 0,9 bits por amostra da ordem 2 para a 3, e é por o mínimo ser intermédio e variável que a ordem é escolhida em cada bloco e em cada canal. Como o codificador avalia as quatro ordens e fica com a mais barata, a escolha adaptativa nunca é pior do que a melhor ordem fixa. O preditor não reinicia nas fronteiras dos blocos, porque as três amostras anteriores vêm do bloco precedente, o que evita uma previsão má no início de cada bloco e torna possível o processamento em fluxo da Secção 4.4.

O tamanho do bloco é o único parâmetro com valor fixo. A Tabela 3 mostra uma curva plana, com 0,84 % entre os extremos testados, e o melhor valor medido é 2048, com 2,2883 de taxa total contra 2,2838 do bloco adotado. Como o tempo de codificação não separa os dois, manteve-se 4096, porque a diferença fica abaixo de um quarto de ponto percentual e é com esse valor que foram obtidas as restantes medições.

Tabela 3: Tamanho do bloco, nos 20 ficheiros de treino, com os restantes parâmetros decididos pelo codificador. O tempo é o total dos 20, em tempo de parede e melhor de três execuções. A negrito, o valor por omissão.

Amostras por bloco	2048	4096	8192	16 384
Taxa de compressão (total)	2,2883	**2,2838**	2,2773	2,2692
Taxa de compressão (média)	2,4973	**2,4903**	2,4787	2,4613
Tempo de codificação (s)	2,07	**2,09**	2,17	2,15

4.3 Codificação entrópica: *BitStream*, Rice e partições

Um ficheiro só guarda bytes, mas a codificação precisa de escrever valores com um número arbitrário de bits, colados uns aos outros e eventualmente partidos entre dois bytes. É essa a função do `BitStream`, que completa o último byte com zeros, razão pela qual o número de *frames* consta do cabeçalho. A primeira versão fazia uma chamada ao `iostream` por byte, três milhões e meio delas num ficheiro comprimido de 3,5 MiB e em cada sentido, e a versão final acumula os bits num registo de 64 bits e os bytes num *buffer* de 64 KiB, sem alterar a ordem dos bits nem o formato. A descodificação desceu de 0,075 s para 0,043 s por ficheiro, valor que inclui o processamento em fluxo por as duas alterações terem entrado na mesma versão.

Os resíduos do preditor são inteiros com sinal, concentrados perto de zero e sem limite superior garantido. O mapeamento zigzag intercala negativos com positivos e converte essa distribuição simétrica numa distribuição aproximadamente geométrica, e para distribuições geométricas o código de Golomb é o código de prefixo de menor comprimento médio, que é a razão de ser o mais adequado aqui. Uma tabela de Huffman teria de ser transmitida ou fixada de antemão e perde eficiência quando um símbolo concentra probabilidade muito alta, que é o caso dos resíduos em trechos silenciosos. Com m potência de dois, a variante de Rice, o quociente e o resto reduzem-se a um deslocamento e a uma máscara, e o custo em bits de um valor u com parâmetro k é $\lfloor u/2^k \rfloor + 1 + k$, expressão que o codificador avalia em cada decisão em vez de uma estimativa de entropia. O k de arranque vem da média dos resíduos em zigzag, a regra do `flac`, e é refinado nos dois vizinhos, porque perto do mínimo a curva de custo é plana.

Um único k por bloco serviria mal um bloco com um ataque percussivo no início e silêncio no resto, pelo que o bloco é dividido em partições, de 1 a 128, cada uma com o seu k e um mínimo de 16 amostras, com o número escolhido por bloco e por canal pelo mesmo critério de custo, ao preço de três bits de expoente mais cinco bits por cada k. O ganho medido é de 2,2416 para 2,2838 na taxa total e de 2,4336 para 2,4903 na taxa média, atribuível às partições porque as restantes alterações da mesma versão foram verificadas como neutras na taxa, com os ficheiros comprimidos pelas duas versões idênticos bit a bit. Com as partições, o modo de canais e a ordem do preditor decididos por bloco, o codificador não tem nenhum parâmetro afinado ao conjunto de desenvolvimento à exceção do tamanho do bloco, o que responde ao aviso da equipa docente sobre especialização. Os dois codecs deste trabalho usam codificadores entrópicos diferentes de propósito, e a Secção 5.3 dá o custo medido de aplicar este código de Rice aos coeficientes da MDCT.

4.4 Processamento em fluxo e paralelismo

A primeira versão carregava o ficheiro inteiro num vetor de inteiros de 32 bits, o dobro do tamanho de um WAV de 16 bits, e lia-o uma segunda vez para localizar o bloco `data`, pelo que a memória crescia linearmente com a duração do áudio. Nada no algoritmo obriga a isso, porque todas as decisões são locais ao bloco e o preditor precisa apenas das três amostras anteriores, e a versão final mantém em memória uma janela com um lote de blocos mais três *frames* de histórico por canal. Nos 20 ficheiros a memória de codificação desceu de 45,6 MiB para 4,8 MiB e a de descodificação de 40,6 MiB para 4,0 MiB, mas o ponto não é a memória ser menor, é deixar de depender da duração do áudio, o que se vê no ficheiro de 44,4 MiB, onde a versão em memória precisa de 157,4 MiB e a versão em fluxo se mantém nos 4,6 MiB. A alteração não custou compressão, e isso foi verificado em separado da alteração das partições, com os ficheiros comprimidos pelas duas versões idênticos bit a bit, pelo que a diferença de taxa da Tabela 4 vem das partições de Rice.

A mesma propriedade torna possível o paralelismo, porque a decisão de cada bloco não depende de nenhuma decisão tomada noutro bloco. Os blocos de cada lote são decididos em paralelo e escritos sempre pela ordem original, o que mantém o formato determinista, com ficheiro idêntico com 1, 2, 4 ou 8 fios de execução, verificado com o `cmp`. No ficheiro de referência de 30 segundos o tempo de parede desce de 0,21 s com um fio para 0,10 s com quatro e 0,07 s com oito, o tempo de CPU sobe para valores entre 0,23 s e 0,27 s, e a memória passa de 4,1 MiB para 5,1 MiB. Por omissão são quatro fios, ou os núcleos disponíveis se forem menos, o que mantém a memória baixa e não pressupõe uma máquina com oito núcleos. O paralelismo ganha em tempo de parede e perde em tempo de CPU, pelo que um único fio é preferível se a avaliação medir tempo de CPU, e o programa permite-o com a opção `--threads 1`.

4.5 Comparação com o estado da arte

A Tabela 5 compara o codec com dois compressores de uso geral e com o `flac` em dois níveis. Contra o `xz -9` e o `zstd -19` a vantagem é folgada, com taxa total 12,2 % e 25,5 % superior, codificação 10 e 6 vezes mais rápida e memória de codificação 37 e 21 vezes menor. O resultado era de esperar, porque nenhum dos dois sabe que está a olhar para áudio, mas serve para situar a ordem de grandeza do que se ganha por modelar o sinal. A exceção está na descodificação, onde o `zstd -19` é cerca de cinco

Tabela 4: Efeito do processamento em fluxo. Tempos médios por ficheiro, memória de pico, taxa total. A diferença de taxa vem das partições de Rice, não do fluxo, que foi verificado como neutro.

			Tempo (s)		Memória (MiB)	
Conjunto	Versão	Taxa	codif.	descodif.	codif.	descodif.
20 ficheiros	em memória	2,2416	0,209	0,075	45,6	40,6
	em fluxo	2,2838	0,098	0,043	4,8	4,0
44,4 MiB	em memória	2,9938	1,54	—	157,4	136,4
	em fluxo	3,1889	0,80	—	4,6	3,9
	`flac -8`	3,4330	0,71	—	3,4	—

vezes mais rápido, comportamento próprio de um formato assimétrico.

Contra o `flac -8` o resultado é desfavorável e convém dizê-lo sem rodeios. A taxa total é 11,1 % inferior, o que equivale a ficheiros comprimidos 12,4 % maiores, duas leituras do mesmo facto que não dão o mesmo número. No tempo de codificação há empate, 0,098 s contra 0,097 s por ficheiro, mas a descodificação é 1,7 vezes mais lenta e a memória 1,3 vezes maior nos dois sentidos. As causas estão identificadas, o `flac` usa predição linear com coeficientes calculados e transmitidos em cada bloco, em vez de preditores fixos, e dispõe de quatro modos de canal em vez de dois, e ambas são retomadas na Secção 6.

A média dos 20 ficheiros esconde duas populações. O SQAM comprime a 2,90 em taxa total e o DSD100 a 1,77, uma diferença de 64 %, porque o SQAM tem instrumentos solo, voz e sinais de teste, com trechos estacionários e silêncios onde o preditor acerta, e o DSD100 tem mixagens densas de banda larga onde sobra pouca redundância. Dar apenas a média juntaria duas populações num número que não descreve nenhuma delas, e o conjunto cego pode ter proporções diferentes. Quanto à exatidão, todos os codecs da tabela reconstroem as amostras sem erro e, nestes ficheiros, reproduzem também o original byte a byte, por não terem blocos de metadados, pelo que a coluna foi retirada da tabela e a diferença discutida na Secção 4.1 só se manifesta em contentores com blocos adicionais.

Tabela 5: Codecs sem perdas nos 20 ficheiros de treino, com as duas convenções de taxa da Secção 2.3. Tempos médios por ficheiro e memória de pico. As linhas indentadas são colecções, não codecs, e a taxa média dos compressores externos não foi registada por ficheiro.

	Taxa		Tempo (s)		Memória (MiB)	
Codec	média	total	codif.	descodif.	codif.	descodif.
`flac -8`	—	2,5675	0,097	0,025	3,6	3,1
`flac -5`	—	2,5343	0,035	0,024	3,6	3,1
O nosso	2,4903	2,2838	0,098	0,043	4,8	4,0
DSD100	1,8485	1,7698	0,089	0,040		
EBU SQAM	3,1320	2,9019	0,106	0,047		
`xz -9`	—	2,0360	0,987	0,146	178,8	14,6
`zstd -19`	—	1,8202	0,618	0,009	101,3	12,1

5 Parte III — Codec de áudio com perdas baseado na DCT

Como o enunciado avalia a distorção pelo SNR, o codec foi desenhado para minimizar o erro quadrático para cada débito, e não o erro percebido. A Figura 3 mostra a cadeia. Em estéreo, os canais passam primeiro para MID/SIDE ortonormal, que concentra a energia no canal comum. Cada canal é então decomposto pela MDCT em blocos de $N = 2048$ coeficientes com sobreposição de metade, o que concentra a energia em poucos coeficientes. Estes são quantizados com um passo Δ único e uma zona morta, que deixa a maioria a zero, e escritos por um codificador aritmético adaptativo com contexto. A quantização é a única etapa com perdas; as restantes são reversíveis e foram escolhidas ortonormais, de modo que o erro introduzido nos coeficientes é igual ao erro no áudio. Assim, todo o compromisso entre débito e distorção é controlado por um único parâmetro, Δ, que o codificador ajusta por bisseção até atingir o débito ou o SNR pedidos. O descodificador percorre a cadeia em sentido inverso, um bloco de cada vez.

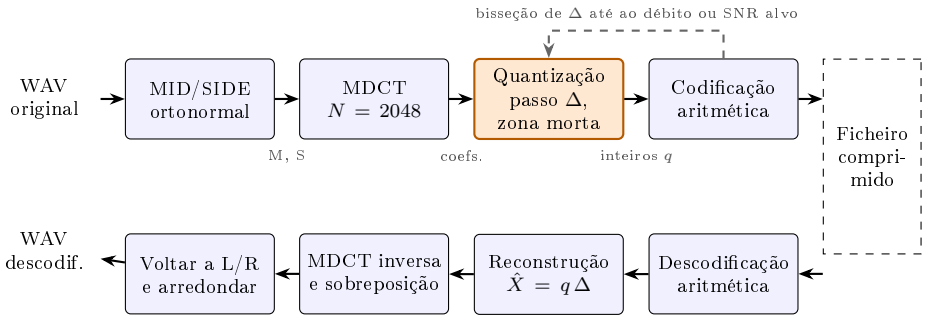


Figura 3: Diagrama de blocos do codec com perdas. Em cima, o codificador (`wav_lossy_enc`); em baixo, o descodificador (`wav_lossy_dec`). A quantização, a laranja, é a única etapa com perdas.

5.1 MDCT: blocos, janela e sobreposição

Aplicar a DCT-II a blocos independentes, como faz o JPEG com blocos de 8×8 píxeis, cria descontinuidades nas fronteiras, porque cada bloco é quantizado separadamente. Usámos por isso a MDCT: blocos de $2N$ amostras com sobreposição de metade, multiplicados por uma janela seno, que produzem N coeficientes cada (amostragem crítica). O *aliasing* de cada bloco é cancelado pelo seguinte (TDAC), e a reconstrução sem quantização é perfeita.

A MDCT é calculada como uma DCT-IV do bloco "dobrado", obtida por uma FFT de $N/2$ pontos, em $O(N \log N)$. Com a janela seno e a escala $\sqrt{2/N}$, a transformada é ortonormal: o erro de quantização nos coeficientes é igual ao erro no áudio, o que permite controlar o SNR no domínio dos coeficientes. Adotou-se $N = 2048$ (janela de 93 ms), escolhido por experiência (Secção 5.4). O `test_mdct` confirma, para N entre 16 e 2048, que a versão rápida coincide com a definição, que a energia se conserva (erro relativo $< 2 \times 10^{-14}$) e que a reconstrução é exata ao nível da precisão numérica.

A janela seno, $w[n] = \sin\big(\pi(n + \frac{1}{2})/2N\big)$, foi escolhida por cumprir a condição de Princen-Bradley, $w[n]^2 + w[n+N]^2 = 1$, que decorre de $\sin^2 + \cos^2 = 1$: é ela que faz o TDAC cancelar o *aliasing* e que torna a transformada ortonormal, e a janela seno é a mais simples que a cumpre. Pela mesma razão, em estéreo usa-se $M = (L+R)/\sqrt{2}$ e $S = (L-R)/\sqrt{2}$: ao contrário do MID/SIDE inteiro e exatamente reversível do codec sem perdas (Secção 4.2), aqui a exatidão inteira não serve, porque os coeficientes vão ser quantizados; o que importa é a ortonormalidade, que mantém o erro igual nos dois domínios. Mantendo o resto do codec e o débito, a MDCT ganha em média 3,17 dB a 64 kbit s^{-1} e 2,44 dB a 128 kbit s^{-1} face à DCT-II em blocos independentes de 2048 amostras. O ganho concentra-se no SQAM (+5,8 e 4,4 dB), gravações com silêncios e ataques isolados, onde as descontinuidades entre blocos pesam mais; nas misturas do DSD100 é de cerca de 0,5 dB, e em 9 dos 40 casos a DCT-II é ligeiramente melhor.

5.2 Quantização dos coeficientes

Cada coeficiente X é quantizado com um passo Δ e reconstruído no descodificador como $\hat{X} = q\,\Delta$, com

$$q = \operatorname{sinal}(X)\,\big\lfloor |X|/\Delta + z \big\rfloor . \tag{2}$$

Esta é a única etapa com perdas do codec. Usa-se o mesmo Δ em todos os coeficientes: como a MDCT e o MID/SIDE são ortonormais, o erro no áudio é a soma dos erros nos coeficientes, e para minimizar essa soma com um dado número de bits o erro deve ser igual em todos eles. É aqui que o codec difere do MP3 e do AAC, que variam o passo por banda de frequência segundo um modelo do ouvido. Um único parâmetro controla assim todo o compromisso entre débito e distorção.

O parâmetro z define uma zona morta: com $z = 0{,}5$ obtém-se o arredondamento ao inteiro mais próximo; com $z < 0{,}5$, valores próximos de um limiar passam para o nível de menor módulo, em particular para zero. Isto aumenta ligeiramente o erro, mas poupa muitos bits, porque o zero é o valor mais barato para o codificador entrópico, e os bits poupados permitem um Δ menor.

Adotou-se $z = 0{,}40$, que ganha 0,35 dB a 64 kbit s^{-1} e 0,28 dB a 128 kbit s^{-1} face ao arredondamento normal (Secção 5.4); com ele ficam a zero, em média, 86 % e 73 % dos coeficientes, respetivamente.

N e z são os únicos parâmetros fixos do codec e foram escolhidos por varredura nos ficheiros de treino, o que exige cuidado com a especialização. Em ambos o máximo é interior e a curva é plana à

sua volta: $z = 0{,}35$ e 0,45 ficam a menos de 0,12 dB do máximo, e $z = 0{,}40$ está a menos de 0,1 dB do melhor valor em 39 dos 40 casos (ficheiro × débito), pelo que não há um pico estreito em que sobreajustar. Entre $N = 1024$ e 2048 a diferença é de 0,03 dB a 128 kbit s^{-1}, mas o melhor N varia muito de ficheiro para ficheiro. Além disso, o critério de escolha foi o SNR, que não penaliza o pré-eco (o espalhamento do erro para antes de um ataque, mais audível com janelas longas): favorece, por isso, janelas mais longas do que as que um critério perceptual escolheria.

5.3 Codificação entrópica e controlo de débito

Após a quantização, a maioria dos coeficientes é nula (73 % a 128 kbit s^{-1} e 86 % a 64 kbit s^{-1}). Por isso não se reutilizou o código de Rice da Parte II: gasta pelo menos um bit por valor, mesmo nulo, e com 88 200 coeficientes por segundo esse bit obrigatório custaria sozinho 88 kbit s^{-1}, mais do que o débito alvo de 64 kbit s^{-1}: o código nem conseguiria atingir o alvo. Com o mesmo Δ, o ficheiro seria em média 3,45 e 2,56 vezes maior (Tabela 6). Usou-se um codificador aritmético binário adaptativo, em que cada decisão custa $-\log_2 p$ bits: um zero previsível custa uma fração de bit. As probabilidades adaptam-se durante a codificação (o descodificador repete as mesmas atualizações) e dependem de 48 contextos: seis zonas de frequência, em escala logarítmica, combinadas com oito níveis de amplitude dos coeficientes vizinhos (os dois anteriores do mesmo bloco e os da mesma frequência no bloco anterior).

O débito é controlado pelo passo Δ: `-q` usa um passo fixo, `-r` procura o passo que atinge um débito (por omissão 128 kbit s^{-1}) e `-s` o que atinge um SNR. A procura estima Δ numa amostra de 48 blocos e afina-o com duas a quatro passagens pelo método da secante, até o débito ficar abaixo do alvo e a menos de 0,05 % dele, sem guardar o ficheiro em memória.

5.4 Justificação experimental das decisões

A Tabela 6 resume as experiências, feitas com o débito fixo e variando um parâmetro de cada vez, mantendo os restantes nos valores adotados. Cada bloco compara apenas valores do mesmo parâmetro, pelo que os números não são comparáveis entre blocos.

Tamanho do bloco N (referência: os outros tamanhos). Em média, $N = 2048$ é o melhor nos dois débitos, mas a vantagem sobre $N = 1024$ é de 0,28 dB a 64 kbit s^{-1} e de apenas 0,03 dB a 128 kbit s^{-1}, que não é significativa; nas misturas do DSD100, $N = 1024$ é ligeiramente melhor. Ficheiro a ficheiro, o melhor N varia muito: nenhum valor fica a menos de 0,1 dB do melhor em mais de 13 dos 40 casos (ficheiro × débito). $N = 2048$ é, portanto, um compromisso médio, e escolher N por ficheiro ganharia em média 0,50 dB e 0,64 dB (trabalho futuro).

Zona morta z (referência: arredondamento normal, $z = 0{,}5$). O valor $z = 0{,}40$ ganha 0,35 dB e 0,28 dB. É também a escolha mais robusta: fica a menos de 0,1 dB do melhor z em 39 dos 40 casos, e o máximo está no mesmo ponto nas duas coleções e nos dois débitos.

Modo de canais (referência: L/R). O MID/SIDE ortonormal ganha 1,44 dB e 1,71 dB, e é melhor ou igual em 38 dos 40 casos.

Codificador entrópico (referência: o codificador aritmético, com o mesmo Δ). Com um código de Rice com o melhor parâmetro por bloco e por canal, o ficheiro seria em média 3,45 vezes maior a 64 kbit s^{-1} e 2,56 vezes a 128 kbit s^{-1} (Secção 5.3).

Para evitar especializar o codec nos ficheiros de treino, como a equipa docente recomenda, escolheram-se valores que são os melhores nas duas coleções e que já o eram nos sete ficheiros preliminares usados no início do trabalho, e o modelo de contexto é adaptativo: aprende cada ficheiro desde o início, sem tabelas treinadas.

5.5 Comparação com MP3 e AAC

O codec foi comparado com o MP3 (codificador LAME) e com o AAC (codificador nativo do `ffmpeg`), em débito constante, a 32, 64, 96, 128, 192 e 256 kbit s^{-1}, nos 20 ficheiros. Os ficheiros descodificados foram acertados ao número de amostras do original e o SNR foi medido com o `wav_cmp`. O alinhamento temporal foi verificado por correlação cruzada: o atraso ótimo é zero para os dois codificadores. Esta verificação é necessária, porque um desfasamento de uma única amostra baixa o SNR do MP3 em 6 a 7 dB, o que inflacionaria artificialmente a nossa vantagem. Quanto ao débito, o MP3 acerta no pedido,

Tabela 6: Parâmetros do codec com perdas, média dos 20 ficheiros, com o valor adotado a negrito. Nos três primeiros blocos os valores são o SNR médio (dB), no último o tamanho relativo ao do codificador aritmético com o mesmo Δ.

Parâmetro	Valor	Débito	
		64 kbit s^{-1}	128 kbit s^{-1}
Coeficientes por bloco, N	512	36,13	45,32
	1024	37,11	45,96
	2048	**37,39**	**45,99**
	4096	36,87	45,31
Zona morta, z	0,30	37,17	45,64
	0,40	**37,39**	**45,99**
	0,50	37,04	45,71
Modo de canais	L/R	35,95	44,28
	MID/SIDE	**37,39**	**45,99**
Codificador entrópico (tamanho relativo)	**aritmético**	**1,00**	**1,00**
	Golomb/Rice	3,45	2,56

mas o AAC fica sistematicamente abaixo (por exemplo, 234 kbit s^{-1} quando se pedem 256), o que o desfavorece um pouco; o nosso fica a menos de 0,05 % do alvo.

O nosso codec tem SNR mais alto em todos os débitos (Tabela 7 e Figura 4): a 128 kbit s^{-1}, 45,97 dB contra 21,64 dB do MP3 e 25,38 dB do AAC. Uma diferença de 24 dB equivale a cerca de 4 bits de precisão efetiva ($\approx$ 6 dB por bit) ou a uma energia do erro cerca de 250 vezes menor. A média esconde uma grande dispersão entre coleções: as gravações do SQAM, instrumentos isolados com muito silêncio, ficam 20 dB a 27 dB acima das misturas densas do DSD100 em todos os débitos. Quanto ao desempenho (Tabela 8), a 128 kbit s^{-1} o nosso codificador é mais rápido do que o MP3 e o AAC e usa cerca de três vezes menos memória, que além disso não depende do tamanho do ficheiro; o descodificador é um pouco mais lento do que o do `ffmpeg`. Face à versão anterior do nosso codec, que guardava o ficheiro inteiro em memória, a codificação ficou 3 vezes mais rápida e a memória desceu de 34 MiB (até 67 MiB) para 5,4 MiB, sem alterar o SNR.

Esta diferença de SNR *não* permite concluir que o nosso codec soa melhor: o SNR favorece-o por construção. O nosso codificador minimiza o erro quadrático, que é exatamente o que o SNR mede, enquanto o MP3 e o AAC distribuem o erro segundo um modelo psicoacústico e colocam-no onde o ouvido não o deteta, o que degrada deliberadamente o SNR em troca de audibilidade. Os espectrogramas confirmam a diferença: a 32 kbit s^{-1}, o MP3 elimina tudo acima de cerca de 5,5 kHz, enquanto o nosso codec mantém conteúdo até cerca de 17 kHz mas com muitos coeficientes anulados, que se ouvem como um som metálico. Num teste de audição a débitos baixos o resultado seria provavelmente o inverso. A comparação serve, por isso, para verificar que o controlo de débito atinge o alvo e que a relação débito–distorção é monótona, e não como medida de qualidade percebida, que exigiria testes de audição ou métricas perceptuais. Note-se ainda que o enunciado refere *distortion measures*, no plural: a métrica da avaliação final pode não ser só o SNR.

Codec	Débito real (kbit/s)	SNR (dB)	t codif. (s)	t descodif. (s)	Memória (MiB)
O nosso	128,0	45,97	0,34	0,05	5,4 / 4,2
versão anterior	128,0	45,98	1,14	0,09	34,4 / 28,9
MP3 (LAME)	128,3	21,64	0,46	0,03	16,0 / 14,8
AAC (ffmpeg)	120,9	25,38	0,61	0,03	16,8 / 15,8

Tabela 8: Desempenho a 128 kbit s^{-1}, média dos 20 ficheiros de treino. A memória é a máxima residente (codificação / descodificação); a versão anterior é a do nosso codec antes de passar a processar em fluxo.

6 Conclusões

O codec sem perdas fica 11,1 % atrás do `flac -8` em taxa total, e as duas causas estão identificadas, a predição linear com coeficientes calculados e transmitidos em cada bloco, que substituiria os preditores

	O nosso				
kbit/s	DSD100	SQAM	média	MP3	AAC
32	20,91	40,41	**30,66**	14,44	8,14
64	26,21	48,54	**37,38**	17,73	16,75
96	30,40	53,85	**42,12**	19,82	20,62
128	33,97	57,98	**45,97**	21,64	25,38
192	39,97	65,38	**52,68**	26,84	31,38
256	45,21	72,64	**58,92**	37,78	36,04

Tabela 7: SNR médio (dB) por débito pedido, nos 20 ficheiros de treino. O AAC fica abaixo do débito pedido (de 30,3 a 234 kbit s^{-1}).

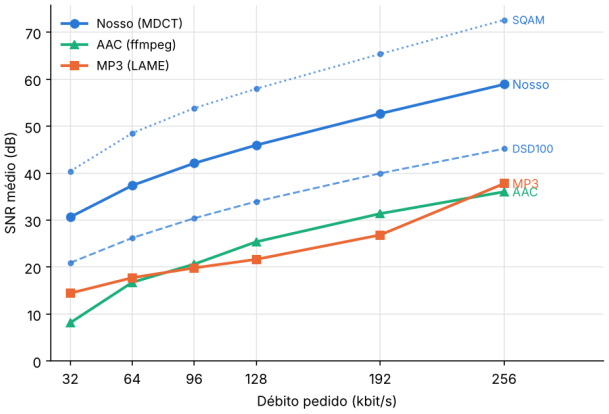


Figura 4: SNR médio em função do débito pedido (média dos 20 ficheiros). A tracejado, as duas colecções que compõem o conjunto. O SNR não mede a qualidade percebida, ver o texto.

polinomiais fixos, e os dois modos de canal que faltam face aos quatro do `flac`. O codec com perdas tem SNR muito superior ao do MP3 e do AAC, mas isso não é uma afirmação sobre qualidade sonora, porque o SNR mede a grandeza que o nosso codec minimiza e não a que os outros minimizam. Nas restantes grandezas o resultado é favorável. O tempo de codificação fica a par do `flac -8`, a memória não depende da duração do áudio, o ficheiro reconstruído é igual ao original byte a byte mesmo com metadados, o paralelismo é determinista, e nenhum parâmetro do codec sem perdas é afinado à mão, à exceção do tamanho do bloco.

A observação que mais ficou não é um número. O crescimento da memória com a duração do áudio só apareceu quando se mediu, já com o codec a funcionar e a comprimir bem, e teria passado despercebido se a avaliação se tivesse ficado pela taxa de compressão. O mesmo se repetiu na Parte III, onde a redução da memória não custou SNR, e na medição dos tempos, que uma ronda feita com o processador em `powersave` chegou a inverter. Medir cedo, medir tudo o que é critério de avaliação, e fixar as condições da máquina antes de comparar.

7 Manual de utilização

A compilação precisa do `build-essential`, do `cmake`, do `pkg-config` e do `libsndfile1-dev`. As comparações com o estado da arte precisam do `flac`, do `zstd`, do `xz-utils` e do `ffmpeg`, e as figuras precisam do `numpy` e do `matplotlib`. A partir da raiz do repositório, dois comandos compilam tudo, `cmake -B build -DCMAKE_BUILD_TYPE=Release` e `cmake --build build -j`, e os executáveis ficam em `build/`. O ficheiro produzido pelo codec sem perdas não depende do número de fios de execução, pelo que a opção `--threads` altera o tempo e não o resultado. Em `scripts/` ficam os *scripts* de apoio, os testes aos 25 casos-limite, as comparações com o estado da arte, o recálculo em `numpy` e as figuras, e na raiz os dois de medição usados neste relatório, `medicoes_relatorio.sh` e `varredura_bloco.py`.

Tabela 9: Invocação dos programas.

Invocação	Efeito
`wav_hist [-k K] [-o f.csv] in.wav canal`	Histograma de um canal, em *bins* de 2^K valores.
`wav_cmp orig.wav mod.wav`	MSE, L^∞ e SNR por canal e para o MID.
`wav_quant -b BITS [-t] in.wav out.wav`	Quantiza para BITS bits, ponto médio ou truncagem.
`wav_effects in.wav out.wav efeito [par.]`	Efeitos `echo`, `multiecho`, `am` e `vdelay`.
`wav_lossless_enc [-b N] [-r] [--threads T] in.wav comp`	Codifica sem perdas, bloco de N amostras, T fios.
`wav_lossless_dec comp out.wav`	Descodifica e reproduz o original byte a byte.
`wav_lossy_enc [-r KBPS | -s DB | -q PASSO] [-n N] [-z R] [-l] in.wav comp`	Codifica com perdas por débito, SNR ou passo fixo.
`wav_lossy_dec comp out.wav`	Descodifica com perdas.